

Connectivity Remediation & LLM Egress Enablement Request

Stage environment · AWS account 087084717211 · us-east-1
Eugene Agent UI (next-gen) & agent back-end

Document	Connectivity remediation report & network-team action request
Prepared by	Rajesh Gupta — Eugene platform engineering
Date	June 06, 2026
Version	1.0
Audience	CSL Network / Cloud-Platform team; Eugene stakeholders
Status	Action required — see §5 (Options) and §7 (Email)

One-line summary

The Eugene next-gen UI and its entire back-end (auth, agent, MCP, Neo4j with **484,259** nodes / **10.7M** relationships) are deployed and healthy. The **only** remaining blocker is network-layer egress from the ECS VPC to an LLM endpoint — a change only the network team can make. Three resolution options are in §5.

1 Executive summary

The Eugene next-generation agent UI was stood up on AWS ECS Fargate behind the internal application load balancer and connected to the existing Eugene back-end. Through a sequence of fixes (§2) the UI now loads, authenticates, reaches the agent, the agent reaches the MCP server, and the MCP server reaches Neo4j — which is fully populated and healthy. Every application and infrastructure layer under our control is working.

A single failure remains: when the agent answers a chat question it must call a Large Language Model. That outbound call fails at the **network layer** — the ECS VPC has no working path to any LLM endpoint (public OpenAI, the internal LLM-Gateway, or the Azure OpenAI private endpoint). This is not an application defect; it requires a network/firewall/DNS change that only the CSL network team can perform.

What we need from the network team

Enable egress from VPC vpc-070f89d8985cafb6 (stage Eugene, 10.88.203.0/24) to **one** LLM endpoint. Three concrete options are detailed in §5; the recommended option is the managed **LLM-Gateway**. A ready-to-send email is in §7.

2 What was fixed (application & deployment)

The following were completed by the Eugene team and require no further action. They are listed so the network team can see the surrounding work is done and the LLM hop is the only gap.

#	Area	Fix applied	Result
1	Container image	Built the Next.js UI image (linux/amd64) and pushed to ECR eugene/agent_ui_nextgen: latest.	Image present; ECS can pull it.
2	ECS task definition	Set PORT=8000 (Next standalone binds the container port), the internal-ALB back-end URLs, and TLS-skip for the self-signed ALB cert.	Task starts & passes health checks.
3	Routing / basePath	Added basePath=/nextgen and prefixed all client fetch() calls so assets and API routes resolve behind the ALB path rule.	UI loads at /nextgen.
4	Load balancer	Target-group health check path set to /nextgen; listener rule added; target healthy.	Service steady & in service.
5	Service lifecycle	Replaced the failed console/CloudFormation service with a clean CLI-managed ECS service on the corrected task definition.	1/1 task running, healthy.
6	Authentication	Production back-end is Entra/OAuth only (the dev /auth/token is disabled). Injected a valid Eugene HS256 JWT for an allow-listed user as EUGENE_STATIC_TOKEN.	UI status = authenticated .
7	Data verification	Confirmed Neo4j connectivity and content from inside the VPC.	484,259 nodes / 10.7M rels.

3 Current architecture & component health

Request path (everything below is healthy except the final LLM hop):

Browser → ALB :80 (/nextgen) → Next.js UI proxy → ALB :443 →
Core API / Agent / MCP → **Neo4j** || Agent → **LLM (BLOCKED)**

Layer	Component	Status	Notes
UI	stage-eugene-agent-ui-nextgen (ECS)	HEALTHY	1/1 task, target healthy at /nextgen
Auth	Eugene JWT (HS256)	HEALTHY	Allow-listed UPN; static token injected
Agent	stage-eugene-agent-ws	HEALTHY	Receives requests; auth + routing OK
MCP	stage-eugene-mcp-ws	HEALTHY	Agent connects (:8443/mcp 200)
Database	Neo4j 10.88.203.251:7687	HEALTHY	484,259 nodes / 10,692,787 rels
LLM	Agent → external LLM	BLOCKED	No network egress from VPC to any LLM endpoint

4 Root cause — LLM egress is blocked at the network layer

The agent (stage-eugene-agent-ws) is configured to call OpenAI (api.openai.com) and the call fails with APIConnectionError after retries — i.e. it cannot open a connection, which is a network problem, not an authentication or key problem. The ECS VPC routes all outbound traffic (0.0.0.0/0) to a Transit Gateway with **no NAT gateway**, and has no DNS resolution for internal CSL zones. Each candidate LLM endpoint is blocked in a different way:

LLM endpoint	Reachability from the ECS VPC	Blocking layer
Public OpenAI api.openai.com:443	Connection never establishes; agent logs show only retries, never a success.	Egress / firewall
LLM-Gateway (API GW) apigtw.dai-platform.cs1g1.cs1g.net	Host name does not resolve (DNS) — the VPC cannot resolve cs1g1.cs1g.net.	DNS
Azure OpenAI (private) 10.206.70.230:443	TCP port opens, but the TLS handshake is reset by peer (source not allow-listed).	Firewall (TLS)

Why this only started recently

The LLM-calling agent first appeared on ECS on **2026-01-16**. Before that, the same workload ran where these endpoints *were* reachable — inside the EKS cluster (where svc.cluster.local resolves) and/or on machines with internet egress. The move to ECS Fargate is what exposed the missing egress path. The LLM logic itself is unchanged and works in those environments.

5 Resolution options for the network team

Enabling **any one** of the following unblocks the agent. They are ordered by our recommendation.

Option A — Managed LLM-Gateway via API Gateway (SigV4)

Recommended

Strategic, supported path: OpenAI-compatible, multi-model (Bedrock), with guardrails and IAM-based auth (no static keys). Designed for “any AWS runtime.”

Network actions required:

- Associate VPC vpc-070f89d8985cafbb6 with Route 53 Resolver rule rslvr-rr-9167b8679b4b4cf7b (already exists, forwards cs1g1.cs1g.net) so the gateway host resolves.
- Authorize the ECS task role stage-uspto_ecs_task_role for execute-api:Invoke and onboard it to the gateway (x-caller-arn path).
- Endpoint to be reached: https://apigtw.dai-platform.us-east-1.aws.cs1g1.cs1g.net/llmg/v1.

Eugene-side work after enablement: point the agent at the gateway base URL with a SigV4-signed client (small, already scoped). **Pros:** no public internet, no static keys, central governance. **Cons:** needs DNS + onboarding; minor agent code change.

Option B — Azure OpenAI private endpoint

Network actions required:

- Allow source VPC `vpc-070f89d8985cafbb6` / subnet `10.88.203.0/24` to reach `10.206.70.230:443` (`deviation-genai.openai.azure.com`). TCP already routes; the TLS handshake is being reset, so this is a firewall source-allow-list change.

Pros: simplest Eugene-side change (api-key header; standard Azure OpenAI). **Cons:** IP-pinned private endpoint, self-signed cert, static subscription key — less governed than the gateway.

Option C — Direct OpenAI egress

- Allow-list `api.openai.com:443` egress from the VPC via the Transit Gateway, or provide a forward-proxy (we set `HTTPS_PROXY` on the agent).

Pros: works with the key already configured; zero code change. **Cons:** least preferred — public internet egress and an external SaaS key; weakest data-governance posture.

Option D — Run the agent inside EKS (no network change)

Re-home the agent into the EKS cluster where the LLM-Gateway `svc.cluster.local` address already resolves. No network ticket, but a larger platform/migration effort; noted for completeness.

Option	Network change	Eugene change	Governance	Recommendation
A · LLM-Gateway	DNS assoc + IAM onboarding	Small (SigV4)	Strong	Preferred
B · Azure OpenAI	Firewall allow-list	Minimal	Medium	Good fallback
C · Direct OpenAI	Egress allow-list / proxy	None	Weak	Least preferred
D · Agent in EKS	None	Large (migration)	Strong	Long-term

6 Evidence & key identifiers

Diagnostics were run from inside the ECS VPC (ephemeral Fargate task on the agent subnet).

Test evidence

Test (from ECS VPC)	Observed result	Conclusion
Agent → OpenAI (7-day logs)	50× "Retrying /chat/completions", 0 success	Egress blocked
DNS resolve gateway hosts	gaierror: Name or service not known	VPC DNS can't resolve cslg1.cslg.net
TCP 10.206.70.230:443	OPEN	Route to Azure PE exists
TLS to 10.206.70.230	ConnectionReset (handshake)	Firewall blocks source
VPC DNS config	AmazonProvidedDNS only; resolver rule not associated	DNS not wired for internal zones
NAT gateways in VPC	none	Egress only via Transit Gateway
Neo4j MATCH (n) count	484,259 nodes / 10,692,787 rels	Database healthy & populated

Key identifiers (for the network team)

Item	Value
AWS account / region	087084717211 (Diffusion-Labs-Dev) / us-east-1
ECS VPC	vpc-070f89d8985cafb6 (CIDR 10.88.203.0/24)
Agent subnet / SG	subnet-0387dd9353f3f91b7 / sg-00e0b6e4e2cfe60b9
Egress route	0.0.0.0/0 → tgw-0122884db8995c4d1 (no NAT)
Route 53 Resolver rule	rslvr-rr-9167b8679b4b4cf7b (FORWARD, not associated to VPC)
ECS task role (SigV4)	arn:aws:iam::087084717211:role/stage-uspto_ecs_task_role
LLM-Gateway (API GW)	https://apigtw.dai-platform.us-east-1.aws.cslg1.cslg.net/llmg/v1
LLM-Gateway (health)	https://llmg.aia.dev.cslg1.cslg.net/v1/health
Azure OpenAI PE	https://10.206.70.230:443 (Host deviation-genai.openai.azure.com)
Neo4j	neo4j+ssc://10.88.203.251:7687

7 Ready-to-send email — Network / Cloud-Platform team

Copy the block below into a new email (adjust recipients as needed).

To	Network & Cloud-Platform Team
Cc	Eugene Platform · <LLM-Gateway onboarding team>
From	Rajesh Gupta
Subject	[Action required] LLM egress for the Eugene agent on ECS (stage, acct 087084717211)

Hi team,

The Eugene next-gen agent UI is deployed and healthy on ECS Fargate (stage, account 087084717211, us-east-1). The UI, authentication, agent, MCP server and Neo4j (484,259 nodes) all work. The one remaining blocker is **outbound egress from the ECS VPC to an LLM endpoint** — the agent cannot reach any model, so chat answers fail.

Diagnostics from inside VPC vpc-070f89d8985cafbb6 (10.88.203.0/24; egress via tgw-0122884db8995c4d1, no NAT) show: OpenAI never connects; the LLM-Gateway host does not resolve in DNS; and the Azure OpenAI private endpoint accepts TCP but resets the TLS handshake. Enabling **any one** of the options below unblocks us. We prefer **Option 1 (LLM-Gateway)**.

Option 1 — LLM-Gateway (preferred): (a) associate VPC vpc-070f89d8985cafbb6 with Route 53 Resolver rule rslvr-rr-9167b8679b4b4cf7b so *.cslg1.cslg.net resolves; and (b) authorize ECS task role stage-uspto_ecs_task_role for execute-api:Invoke / onboard it to the gateway (https://apigtw.dai-platform.us-east-1.aws.cslg1.cslg.net/llmg/v1).

Option 2 — Azure OpenAI: allow source 10.88.203.0/24 to reach 10.206.70.230:443 (deviation-genai.openai.azure.com) — TCP already routes, the TLS handshake is being reset, so a firewall source allow-list is needed.

Option 3 — Direct OpenAI: allow-list api.openai.com:443 egress for the VPC via the Transit Gateway, or provide a forward-proxy host:port we can configure.

Could you advise which option is acceptable and action it (or tell me the owning team)? I'm happy to jump on a quick call. A detailed report with full evidence and identifiers is attached.

Thanks,

Rajesh Gupta — Eugene platform engineering

Prepared by Rajesh Gupta · Eugene platform engineering · June 06, 2026 · Confidential — Internal use only